

# Setup Environment

## Setting Up the Environment on Windows

**Workflow:** Windows CMD → WSL 2 → Ubuntu → APT → Python Virtual Environment → PyTorch

### 1. Overview

Our goal is to establish a local Linux-based development environment for machine learning and scientific computing without replacing Windows.

| Component   | Description                                                                  |
|-------------|------------------------------------------------------------------------------|
| Windows CMD | Command-line interface used to launch WSL                                    |
| WSL 2       | Runs a real Linux kernel using lightweight virtualization                    |
| Ubuntu      | Linux distribution providing the development environment                     |
| APT         | Package manager for installing and updating system software                  |
| Python      | Programming language used for machine learning                               |
| venv        | Creates isolated Python environments                                         |
| pip         | Installs Python packages                                                     |
| PyTorch     | Deep learning framework for tensor computation and automatic differentiation |
| Docker      | Optional containerization platform for reproducible environments             |

### 2. Installing WSL 2 and Ubuntu

#### Step 1 — Check WSL

Open Windows CMD or PowerShell:

```
wsl --list --verbose
```

Initially, our system displayed:

| NAME             | STATE   | VERSION |
|------------------|---------|---------|
| * docker-desktop | Stopped | 2       |

This indicated that Docker Desktop had already created a WSL 2 distribution, but Ubuntu was not yet installed.

## Step 2 — Install Ubuntu

Run:

```
wsl --install -d Ubuntu
```

The installer downloads Ubuntu, registers the distribution, and initializes the Linux environment.

During the first launch, Ubuntu asks you to create a Unix username and password.

**Important:** Linux does not display characters or asterisks when entering passwords. This is normal.

## Step 3 — Verify Installation

From Windows CMD or PowerShell:

```
wsl -l -v
```

Our system subsequently showed:

| NAME             | STATE   | VERSION |
|------------------|---------|---------|
| * docker-desktop | Stopped | 2       |
| Ubuntu           | Running | 2       |

The `VERSION 2` field confirms that Ubuntu uses WSL 2.

Set Ubuntu as the default distribution:

```
wsl --set-default Ubuntu
```

Launch Ubuntu:

```
wsl -d Ubuntu
```

## 3. Understanding the Linux Terminal

After launching Ubuntu, our terminal prompt looked like:

```
lawson@damn:~$
```

The components mean:

- `lawson` : Linux username.
- `damn` : Computer hostname.
- `~` : Current user's home directory.
- `$` : Regular-user shell prompt.

Useful Linux commands:

| Command            | Meaning                 | Purpose                                   |
|--------------------|-------------------------|-------------------------------------------|
| <code>pwd</code>   | Print Working Directory | Display current directory                 |
| <code>ls</code>    | List                    | List files and folders                    |
| <code>cd</code>    | Change Directory        | Navigate directories                      |
| <code>mkdir</code> | Make Directory          | Create a directory                        |
| <code>touch</code> | Touch                   | Create an empty file or update timestamps |
| <code>cat</code>   | Concatenate             | Display or combine file contents          |
| <code>sudo</code>  | Superuser Do            | Execute commands with elevated privileges |

Return to the Linux home directory:

```
cd ~  
pwd
```

Expected result:

```
/home/lawson
```

**Note:** Windows drives are accessible inside WSL through paths such as `/mnt/c/`.

## 4. Installing System Development Tools with APT

APT (Advanced Package Tool) is a package management tool used to install, update, upgrade, and remove software packages on Debian-based Linux distributions.

Refresh package information:

```
sudo apt update
```

Upgrade installed packages:

```
sudo apt upgrade -y
```

Install development tools:

```
sudo apt install -y git python3 python3-pip python3-venv
```

Verify installation:

```
python3 --version  
git --version
```

### APT vs pip:

- APT manages system-level software packages.
- pip manages Python packages, preferably inside virtual environments.

## 5. Creating the Research Project

We created a dedicated project directory inside Ubuntu:

```
cd ~  
mkdir -p research/first-neural-network  
cd research/first-neural-network
```

Verify the location:

```
pwd
```

Expected result:

```
/home/lawson/research/first-neural-network
```

This directory stores our Python code, virtual environment, and training results.

## 6. Creating a Python Virtual Environment

A Python virtual environment isolates dependencies for a specific project.

Create the environment:

```
python3 -m venv .venv
```

Activate it:

```
source .venv/bin/activate
```

The terminal prompt changes to:

```
(.venv) lawson@damn:~/research/first-neural-network$
```

The `(.venv)` prefix indicates that the virtual environment is active.

To deactivate it later:

```
deactivate
```

The environment remains on disk even after closing the terminal.

## 7. Installing PyTorch

With the virtual environment activated:

```
python -m pip install torch
```

Check the installed version:

```
python -c "import torch; print(torch.__version__)"
```

Test tensor computation:

```
python -c "import torch; print(torch.rand(2, 2))"
```

Our test successfully generated a random 2×2 tensor, confirming that PyTorch could perform basic tensor operations.

However, we also encountered:

```
UserWarning: Failed to initialize NumPy
No module named 'numpy'
```

Install the missing dependency:

```
python -m pip install numpy
```

Verify both libraries:

```
python -c "import torch, numpy; print('PyTorch:', torch.__version__);
print('NumPy:', numpy.__version__)"
```

For this initial experiment, CPU-based training is sufficient. GPU acceleration can be configured separately.

## 8. Creating and Running Python Files

Create a Python file:

```
touch train.py
```

Or open it for editing:

```
nano train.py
```

Run the Python program:

```
python train.py
```

We initially attempted:

```
torch train.py
```

This failed because **PyTorch is a Python library, not a command-line interpreter**.

The correct workflow is:

```
Linux Terminal
  ↓
Python Interpreter
```

```
↓  
train.py  
↓  
PyTorch Library  
↓  
Neural Network Computation
```

## 9. Troubleshooting

### Problem A — `wsl -l -v` fails inside Ubuntu

We received an error from an unrelated Linux program named `wsl`.

Reason: `wsl` without `.exe` inside Ubuntu resolved to a different executable.

Solution:

```
wsl.exe -l -v
```

Alternatively, return to Windows:

```
exit
```

Then run:

```
wsl -l -v
```

### Problem B — Closing CMD accidentally

Closing the terminal does not delete Ubuntu, project files, or installed dependencies.

Reopen CMD:

```
wsl -d Ubuntu
```

Return to the project:

```
cd ~/research/first-neural-network
```

Reactivate the environment:

```
source .venv/bin/activate
```

An active foreground training process may terminate when its terminal closes, but files already saved to disk remain available.

## Problem C — Docker Desktop shows Stopped

Docker does not need to run for ordinary Ubuntu, Python, or PyTorch development.

Docker Desktop is only required when using its container engine to build or run containers.

## Problem D — Terminal split into two panes

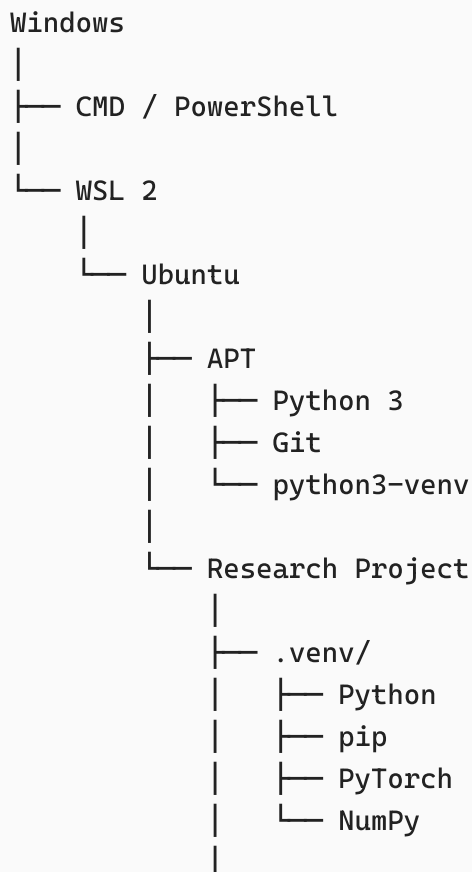
Windows Terminal supports multiple panes.

To close the unwanted pane, select it and use:

```
Ctrl + Shift + W
```

Alternatively, enter `exit` in that pane.

## 10. Final Environment Architecture



```
├── train.py
└── Training Results
```

## 11. Quick Start for Future Sessions

Once the environment has been installed, we do not need to repeat the installation process.

### Windows CMD:

```
wsl -d Ubuntu
```

### Ubuntu:

```
cd ~/research/first-neural-network
source .venv/bin/activate
python train.py
```

These commands restore the project environment and execute the training script, provided `train.py` contains executable training code.

## 12. Key Takeaways

1. WSL 2 provides the infrastructure to run Linux on Windows.
2. Ubuntu provides the Linux operating environment.
3. APT manages system software packages.
4. Python virtual environments isolate project dependencies.
5. pip installs Python libraries such as PyTorch and NumPy.
6. PyTorch performs tensor computations and supports neural network training.
7. Docker is optional and can improve environment portability and reproducibility.
8. Closing the terminal does not erase installed software or saved project files.

**Current progress:** Ubuntu installed, WSL 2 verified, Python virtual environment activated, and PyTorch tensor computation successfully tested.

**Next milestone:** Implement and train the first neural network, evaluate its predictions, and save its learned parameters.